

常量

① 字符串型常量

遇单引号、双引号，要用 \ 将其转义为普通字符，如 \" ' 等 ("Kitty's" 也是可以的)

② 数值型常量

十进制型直接书写，如 -20+10

浮点型用科学记数法表示，如 38.4E-1

二进制型表示为 0b 二进制数，如 0b111001, b'111001' ASCII码 → 9

十六进制型表示为 0x 十六进制数，如 0x2A

③ 日期和时间型常量

日期型常量，包括年月日 SELECT CURDATE();

```
CREATE TABLE operation  
( cid CHAR(6), 日期时间型  
  ctime DATETIME);  
INSERT INTO operation VALUES('01', NOW());  
SELECT * FROM operation;
```

时间型常量，包括小时数、分钟数、秒数、微秒数 SELECT CURTIME();

日期时间型常量，包括时间和日期 SELECT NOW();

④ 布尔型常量

TRUE FALSE

⑤ NULL

无类型常量，表示“没有值”“无数据”“不确定”

与数值0、长度为0的字符串不同

变量

系统变量

分为全局系统变量与会话系统变量

全局系统的变量的修改会影响整个服务器，会话系统的变量的修改会影响当前正在进行的会话

查看系统变量

```
SHOW GLOBAL VARIABLES LIKE '%auto%';  
SESSION
```

查看系统变量值

```
SELECT @@ GLOBAL. AUTO COMMIT ;  
@@ SESSION. (@@)
```

赋值系统变量 (对SESSION的赋值不影响GLOBAL的)
或 DEFAULT

```
SET @@ GLOBAL. AUTO COMMIT = 1 ;  
@@ SESSION. (@@)
```

用户变量

属于某特定会话

① SET 赋值

```
SET @a=10, @b=20;
```

```
SELECT @a, @b;
```

```
SET @v_sno = (SELECT sno FROM student WHERE sname = '王雁');  
结果唯一性
```

② SELECT 赋值

不可缺少，少了就成了判断两边是否相等，返回1或0

```
SELECT @num := (SELECT COUNT(*) FROM student);
```

或 SELECT COUNT(*) INTO @num FROM student;

```
SELECT 45+500 INTO @num;
```

```
SELECT @num;
```

局部变量

作用范围仅限于语句块 (BEGIN... END)

定义

```
DECLARE 变量名 数据类型 DEFAULT 默认值;
```

赋值

没有就默认为 NULL

```
DECLARE var-s CHAR(3) DEFAULT '女';
```

```
SET var-s = '男';
```

```
DECLARE var_name VARCHAR(10);
```

```
SET var_name = (SELECT sname FROM student WHERE sno = '101');
```

或 SELECT sname INTO var_name FROM student WHERE sno = '101';

注释

单行: -- 或 #

多行: /* ... */

DELIMITER

MySQL 语句执行结束的默认标记符为 ;

用 DELIMITER 可重新定义, 让解释器遇新标记符才整体运行代码,
存储程序执行完后再用 DELIMITER 改回默认的;

语句块

BEGIN... END 语句块不能单独运行, 须包含在存储过程、存储函数、触发器或事件中。

分支结构

IF

将学号为"001108", 课程号为"102"的学生成绩以等级制显示
90~100为A等, 80~89为B等, 60~79为C等, 低于60分为D等

DELIMITER //

CREATE PROCEDURE dj()

BEGIN

DECLARE classification CHAR DEFAULT 'A';

DECLARE v-score INT;

SELECT grade INTO v-score FROM score WHERE sno='001108' AND cno='102';

IF v-score < 60 THEN SET classification = 'D';

ELSE IF v-score < 80 THEN SET classification = 'C';

ELSE IF v-score < 90 THEN SET classification = 'B';

ELSE SET classification = 'A';

END IF;

SELECT v-score, classification;

END//

CALL dj();

匿名块
放入存储过程或存储函数才可执行

CASE

SELECT sno, cno,

CASE

WHEN grade < 60 THEN 'D'

WHEN grade >= 60 AND grade < 79 THEN 'C'

WHEN grade >= 80 AND grade < 90 THEN 'B'

WHEN grade >= 90 THEN 'A'

END AS '总评'

FROM score WHERE sno='001108' AND cno='102';

查询 student 表中学生的系别情况, 若系别的值是 "计算机", 结果显示为 "CS"; 若是 "通信工程", 则显示为 "TE"

```
SELECT sno,  
CASE dept  
  WHEN '计算机' THEN 'CS'  
  WHEN '通信工程' THEN 'TE'  
END AS 系别  
FROM student;
```

根据系别代码输出学院名称. "CS" 输出 "计算机学院", "MS" 输出 "数学学院", 其他输出 "学院代码有误".

```
BEGIN  
  DECLARE v-dept CHAR(2);  
  DECLARE xynome VARCHAR(10);  
  SET v-dept = 'CS';  
  CASE  
    WHEN v-dept = 'CS' THEN SET xynome = '计算机学院';  
    WHEN v-dept = 'MS' THEN SET xynome = '数学学院';  
    ELSE SET xynome = '学院代码有误';  
  END CASE;  
  SELECT xynome;  
END
```

循环结构

WHILE (当型)

计算1~100所有自然数的累加和

```
BEGIN
  DECLARE v-count INT DEFAULT 0;
  DECLARE v-sum INT DEFAULT 0;
  WHILE v-count <= 100 DO
    SET v-sum = v-sum + v-count;
    SET v-count = v-count + 1;
  END WHILE;
  SELECT v-sum;
END
```

REPEAT (直到型)

计算1~100所有能被3整除的自然数的累加和

```
BEGIN
  DECLARE v-number INT DEFAULT 1;
  DECLARE v-sum INT DEFAULT 0;
  REPEAT
    IF v-number % 3 = 0
      THEN SET v-sum = v-sum + v-number;
    END IF;
    SET v-number = v-number + 1;
  UNTIL v-number > 100;
  END REPEAT;
  SELECT v-sum;
END
```

LOOP (无条件, 死循环, 应与跳出循环语句结合使用)

LEAVE ($\approx$ break)

```
BEGIN
  DECLARE v-count INT DEFAULT 0;
  DECLARE v-sum INT DEFAULT 0;
  label: LOOP
    SET v-sum = v-count + v-sum;
    SET v-count = v-count + 1;
    IF v-count > 100
      THEN LEAVE label;
    END IF;
  END LOOP label;
  SELECT v-sum;
END
```

ITERATE ($\approx$ continue)

```
BEGIN
  DECLARE v-number INT DEFAULT 1;
  DECLARE v-sum INT DEFAULT 0;
  label: LOOP
    IF v-number > 100
      THEN LEAVE label;
    ELSEIF MOD(v-number, 3) != 0
      THEN ITERATE label;
    END IF;
    SET v-sum = v-sum + v-number;
    SET v-number = v-number + 1;
  END LOOP label;
  SELECT v-sum;
END
```

游标 (从结果集中逐一读取、处理记录)

定义游标

DECLARE 游标名 CURSOR FOR
SELECT ... ;



打开游标

OPEN 游标名;



使用游标(读取数据)

FETCH 游标名 INTO 变量名;



关闭游标

CLOSE 游标名;

赋值给指定变量

个数、类型应与SELECT查询的相同

根据学号统计某同学所获总学分, 若课程成绩小于70分则该课学分无法计入

DELIMITER \$\$

CREATE PROCEDURE cursor10 (v_sno CHAR(6))

BEGIN

DECLARE sum_credit, v_credit, v_grade INT;

DECLARE v_done INT DEFAULT FALSE;

DECLARE cs_credit_grade CURSOR FOR

SELECT credit, grade

FROM course JOIN score ON course.cno = score.cno

WHERE sno = v_sno;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

SET sum_credit = 0;

OPEN cs_credit_grade;

label : LOOP

FETCH cs_credit_grade

INTO v_credit, v_grade;

IF v_done = TRUE THEN LEAVE label;

END IF;

IF v_grade >= 70 THEN SET sum_credit = sum_credit + v_credit;

END IF;

END LOOP label;

CLOSE cs_credit_grade;

SELECT v_sno, sum_credit;

END \$\$

DELIMITER;

DELIMITER \$\$

CREATE PROCEDURE cursor10(v_sno CHAR(6))

BEGIN

DECLARE sum_credit, v_credit, v_grade INT;

DECLARE v_done INT DEFAULT FALSE;

DECLARE cs_credit_grade CURSOR FOR

SELECT credit, grade FROM course JOIN score ON course.cno = score.cno WHERE sno = v_sno;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = TRUE;

SET sum_credit = 0;

OPEN cs_credit_grade;

label: LOOP

FETCH cs_credit_grade INTO v_credit, v_grade;

IF v_done

THEN LEAVE label;

END IF;

IF v_grade >= 70

THEN SET sum_credit = sum_credit + v_credit;

END IF;

END LOOP label;

CLOSE cs_credit_grade;

SELECT v_sno, sum_credit;

END \$\$

DELIMITER;

存储过程

① 创建与调用

创建存储过程 pro-del-sc, 依学号、课程号删除score表中的数据, 学号和课程号由存储过程的参数传入

```
DELIMITER //
CREATE PROCEDURE pro-del-sc (v-sno CHAR(6), v-cno CHAR(3))
BEGIN
    DELETE FROM score WHERE sno=v-sno AND cno=v-cno;
END //
DELIMITER ;
CALL pro-del-sc ('001106', '101');
```

创建存储过程 pro-inquire, 通过学生姓或名实现模糊查询, 查询符合条件的学号、姓名, 存储过程的参数为学生的姓或名

```
DELIMITER //
CREATE PROCEDURE pro-inquire (v-name CHAR(20))
BEGIN
    SELECT sno, sname FROM student WHERE sname LIKE CONCAT('%', v-name, '%');
END //
DELIMITER ;
CALL pro-inquire ('杨');
```

② 异常处理 - 条件处理器

先对存储程序执行中可能出现的异常进行声明, 再对异常进行适当处理,

保证存储程序在系统警告或报错情况下继续执行, 避免程序因异常而停止, 以顺利调用存储过程

定义条件

外键约束条件异常

```
DECLARE error-foreignkey CONDITION FOR 1452;
```

或

```
DECLARE error-foreignkey CONDITION FOR SQLSTATE '23000';
```

定义处理程序

异常发生时, 执行 statement 后继续执行下一条 SQL 语句

```
DECLARE CONTINUE HANDLER FOR
EXTI
```

异常发生时, 执行 statement 后退出当前语句块

SQLSTATE '值' Statement;

异常码

异常名 (定义条件中的)

SQLWARNING - 01 开头的 SQLSTATE 码对应的异常

NOT FOUND - 02 开头

SQLException - 不以 01 或 02 开头

③ 查看存储过程

查看定义

```
SHOW CREATE PROCEDURE
```

存储过程名

查看状态特征

```
SHOW PROCEDURE STATUS LIKE '匹配存储过程名的模式字符串%';
```

④ 删除存储过程 DROP PROCEDURE 存储过程名;

创建存储过程 insert_sc, 该存储过程将完成向 score 表插入数据, 学号、课程号由输入参数给出, 同时程序返回是否插入成功的结果

```
DELIMITER //
CREATE PROCEDURE insert_sc (v_sno CHAR(6), v_cno CHAR(3), v_grade INT, v_message CHAR(100))
BEGIN
    DECLARE v_done BOOLEAN DEFAULT 0;
    DECLARE error_foreignkey CONDITION FOR 1452;
    DECLARE CONTINUE HANDLER FOR error_foreignkey SET v_done = 1;
    INSERT INTO score VALUES (null, v_sno, v_cno, v_grade);
    IF v_done = 1 THEN SET v_message = 'error-foreignkey';
    ELSE SET v_message = 'success';
    END IF;
END //
DELIMITER ;
→ CALL insert_sc('001107', '209', '60', @message);
SELECT @message;
```

创建 cno_number 表用于存储每门课程选修的人数, 创建存储过程 pro_cno_num 用于统计每门课程选修的人数, 并把课程号、课程名、选修课程人数插入 cno_number 表

```
CREATE TABLE cno_number (
    id INT UNSIGNED AUTO-INCREMENT PRIMARY KEY,
    cno CHAR(3),
    cname VARCHAR(30),
    number INT UNSIGNED);
DELIMITER //
CREATE PROCEDURE pro_cno_num()
BEGIN
    DECLARE v_cno CHAR(3);
    DECLARE v_cname VARCHAR(30);
    DECLARE v_number INT DEFAULT 0;
    DECLARE v_done BOOLEAN DEFAULT 0;
    DECLARE youbiao CURSOR FOR SELECT cno, cname FROM course;
    DECLARE CONTINUE HANDLER FOR SQLSTATE '02000' SET v_done = 1;
    DELETE FROM cno_number;
    OPEN youbiao;
    WHILE v_done DO
        FETCH youbiao INTO v_cno, v_cname;
        SELECT COUNT(*) FROM course WHERE cno = v_cno AND cname = v_cname;
        INTO v_number
        INSERT INTO cno_number VALUES (NULL, v_cno, v_cname, v_number);
    END WHILE;
    CLOSE youbiao;
END //
DELIMITER ;
→ CALL pro_cno_num();
SELECT * FROM cno_number;
```

存储函数

- 存储过程的参数传递类型可为 IN, OUT, INOUT, 存储函数只有 IN 类型
- 存储过程要用 CALL 语句调用, 存储函数可像内置函数一样直接使用
- 存储过程无 RETURN 语句 (不能有返回值), 存储函数必包含 RETURN 语句且只能返回一个值
- 存储过程可调用存储函数而存储函数不可调用存储过程
- 存储过程主要用于执行并完成某个功能, 存储函数主要用于计算并返回一个函数值

① 创建与调用

创建存储函数 fun-st-num, 统计 student 表中 某个系别 的人数, 系名由函数给出

```
DELIMITER //  
CREATE FUNCTION fun-st-num (v_dept VARCHAR(20))  
RETURNS INT  
READS SQL DATA  
COMMENT '系别的人数'  
BEGIN
```

READS SQL DATA 有 SELECT 无 UPDATE

MODIFIES SQL DATA 有 UPDATE

DETERMINISTIC 存储对同样的输入参数产生相同结果

```
    DECLARE v-number INT DEFAULT 0;  
    SELECT COUNT(*) INTO v-number FROM student WHERE dept = v_dept;  
    RETURN v-number;
```

② 查看

```
END//  
DELIMITER;
```

→ SELECT fun-st-num('计算机');

查看定义: SHOW CREATE FUNCTION 存储函数名;

查看状态特征: SHOW FUNCTION STATUS LIKE '匹配存储函数名的模式字符串';

③ 删除

DROP FUNCTION 存储函数名;

存储函数

根据学号统计某同学获得的总学分，若成绩不达70分则该门课程学分不能计入

DELIMITER //

CREATE FUNCTION XF(V-SNO CHAR(6))

RETURNS INT

DETERMINISTIC

BEGIN

DECLARE sum-credit, v-credit, v-grade INT;

DECLARE v-done BOOLEAN DEFAULT 0;

DECLARE cs-credit-grade CURSOR FOR

SELECT credit, grade FROM course JOIN score ON course.cno = score.cno WHERE sno = v-sno;

SET sum-credit = 0;

OPEN cs-credit-grade;

~~BEGIN~~

label: LOOP

FETCH cs-credit-grade INTO v-credit, v-grade;

IF v-done THEN LEAVE LOOP;

END IF;

IF v-grade $\geq$ 70 THEN ^{SET} v-sum-credit = sum-credit + v-credit;

END IF;

END LOOP label;

CLOSE cs-credit-grade;

RETURN sum-credit;

END //

DELIMITER;

触发器 (基于数据表操作)

一种特殊存储过程, 当特定事件发生时由系统自动调用执行, 无须显示执行, 用于监控和响应

触发器不接收任何参数. (存储过程需显示调用, 可接收、传回参数)

触发事件: 插入 INSERT、修改 UPDATE、删除 DELETE

触发对象: 哪个数据表

触发时机: BEFORE、AFTER

触发级别: FOR EACH ROW

- 触发器和指定的数据表关联, 当数据表被删除时, 任何与该数据表相关的触发器也被删除

- 行级触发器执行顺序: 执行 BEFORE 行级触发器 → 执行 DML 语句 → 执行 AFTER 行级触发器

① 创建与调用

创建 delsc 表, 存储从 score 表中删除的学号、课程号、成绩, 从 score 表删除记录时,

触发 triggerdel-sc 触发器向 delsc 表中插入删除的记录

```
CREATE TABLE delsc
```

```
AS SELECT sno, cno, grade FROM score WHERE 1=2; // 只复制结构而不复制数据  
永远不成立
```

```
DELIMITER //
```

```
CREATE TRIGGER triggerdel-sc
```

```
AFTER DELETE ON score
```

```
FOR EACH ROW
```

```
BEGIN
```

```
INSERT INTO delsc (sno, cno, grade) VALUES (OLD.sno, OLD.cno, OLD.grade);
```

```
END//
```

```
DELIMITER;
```

```
DELETE FROM score WHERE sno = '1001';
```

```
SELECT * FROM delsc;
```

② 查看

→ SHOW TRIGGERS;

→ SELECT * FROM INFORMATION-SCHEMA.TRIGGERS;

特定 → 使用 SELECT 语句查看 INFORMATION-SCHEMA 数据库中有关 SCORE 表的触发器

```
SELECT * FROM INFORMATION-SCHEMA.TRIGGERS
```

```
WHERE EVENT-OBJECT-TABLE = 'SCORE';
```

③ 删除

MySQL 无修改触发器的语句, 只能删除 (再重建)

```
DROP TRIGGER IF EXISTS 数据库名 触发器名;
```

创建日志表 score-log (id, operate-date, operate-user, log-text) 用于记录对成绩表所做修改操作的相关信息, 创建触发器 trig-score, 当用户对 score 表操作时将触发 trig-score 触发器. 将操作用户的用户名、操作日期和修改前后的数据插入日志表 score-log.

```
CREATE TABLE score-log
( id INT NOT NULL UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  operate-date DATE,
  operate-user VARCHAR(20),
  log-text VARCHAR(200));
DELIMITER //
CREATE TRIGGER trig-score
AFTER UPDATE ON score FOR EACH ROW
BEGIN
  INSERT INTO score-log (id, operate-date, operate-user, log-text)
  VALUES (NULL, NOW(), USER(), CONCAT('学号为', OLD.sno, '的成绩', OLD.grade,
  '修改为', NEW.grade));
END //
DELIMITER ;

→ UPDATE score SET grade = 80 WHERE sno = '001103' AND cno = '1002';

SELECT * FROM score-log;
```

Student 表和 score 表是关联的, score 表创建时若未对外键设置级联删除, 那么删除 student 表中记录时只要 score 表中有该生成成绩记录, 就无法删除 student 表中的记录

现采用创建触发器 del-sc 来实现删除 student 表中记录时一起删除 score 表中相应记录

```
DELIMITER //
CREATE TRIGGER del-sc
★ BEFORE DELETE ON student
FOR EACH ROW
BEGIN
  DELETE FROM score WHERE sno = OLD.sno;
END //
DELIMITER ;

→ DELETE FROM student WHERE sno = '001104';

SELECT * FROM score WHERE sno = '001104';
```

创建触发器 sum_credit, 实现对 student 表总学分的计算, 当向 score 中添加记录时, student 表总学分的值做相应改变. 当课程成绩大于等于 60 分时, 将课程学分加到该学生总学分中

```
DELIMITER $$
```

```
DROP TRIGGER IF EXISTS sum_credit $$
```

```
CREATE TRIGGER sum_credit
```

```
AFTER INSERT ON score
```

```
FOR EACH ROW
```

```
BEGIN
```

```
    UPDATE student
```

```
    SET totalcredit = totalcredit + 已插入的新记录的课程学分
```

```
    (SELECT credit FROM course WHERE cno=new.cno)
```

```
    WHERE sno = new.sno AND new.grade >= 60;
```

```
END $$
```

```
DELIMITER;
```

```
UPDATE student
```

```
SET totalcredit = totalcredit +
```

```
(SELECT credit FROM course WHERE cno=NEW.cno)
```

```
WHERE sno = NEW.sno AND NEW.grade >= 60;
```

事件 (基于时间被调度)

是一种特殊存储过程, 没有调用者, 被激活后执行

时间周期是其特有属性, 具实时优势

① 开启事件调度器

SET @@ GLOBAL. event-scheduler = ON; (查看是否开启: SHOW VARIABLES LIKE '%event-scheduler%')

② 创建与调用

创建事件 event-score, 该事件的任务为调用存储过程 pro-cno, 实用每月查询每门课程的选修人数、课程平均分、课程最高分、课程最低分, 并把查询结果插入 month-score 表, month-score 表中的字段有 id、cno、number、avggrade、maxgrade、mingrade、updatetime.

```
(1) CREATE TABLE month-score
( id INT UNSIGNED AUTO-INCREMENT PRIMARY KEY,
  cno CHAR(3),
  number INT UNSIGNED,
  avggrade DECIMAL(4,1),
  maxgrade DECIMAL(4,1),
  mingrade DECIMAL(4,1),
  updatetime TIMESTAMP);
```

```
(2) DELIMITER //
CREATE PROCEDURE pro-cno()
BEGIN
  INSERT INTO month-score (id, cno, number, avggrade, maxgrade, mingrade, updatetime)
  SELECT NULL, cno, COUNT(*), AVG(grade), MAX(grade), MIN(grade), NOW()
  FROM score
  GROUP BY cno;
END //
DELIMITER ;
```

```
(3) DELIMITER //
CREATE EVENT event-score
ON SCHEDULE EVERY 1 MONTH DO
BEGIN
  CALL pro-cno();
END //
DELIMITER ;
```

→ SELECT * FROM month-score;

③ 查看

- 所有事件 : SHOW EVENTS;
- 指定事件 : SHOW CREATE EVENT 事件名 ;

④ 修改

修改 event_score, 将每隔一月改为每隔一周, 将事件名称改为 week_score
并未其添加注释 "每个星期统计一次".

```
ALTER EVENT event_score  
ON SCHEDULE EVERY 1 WEEK  
RENAME TO week_score  
COMMENT '每个星期统计一次' (DO...);
```

⑤ 删除

```
DROP EVENT IF EXISTS 数据库名 事件名;
```

针对教学数据库编写一个存储过程,功能是根据学生的学号显示选修的课程名和成绩,

要求返回取得的学分总数和未取得的学分总数(使用游标),参数包括学生学号、取得学分总数和未取得学分总数 ~~★~~ 有价值

DELIMITER //

CREATE PROCEDURE getinfo (IN v_sno CHAR(6), OUT v_getcredit INT, OUT v_notgetcredit INT)
BEGIN

DECLARE v_cname VARCHAR(50);

DECLARE v_grade DECIMAL(5,2);

DECLARE v_credit, v_getcredit, v_notgetcredit INT;

DECLARE v_done BOOLEAN DEFAULT 0; cname,

DECLARE youbiao CURSOR FOR SELECT v_credit, grade FROM course JOIN score ON course.cno = score.cno

DECLARE CONTINUE HANDLER FOR NOT FOUND SET v_done = 1; WHERE sno = v_sno;

SET v_getcredit = 0; SET v_notgetcredit = 0;

OPEN youbiao;

label: LOOP

FETCH youbiao INTO v_cname, v_credit, v_grade;

IF v_done THEN LEAVE LOOP; END IF;

SELECT v_cname, v_grade;

IF v_grade >= 60 THEN SET v_getcredit = v_getcredit + v_credit;

ELSE v_notgetcredit = v_notgetcredit + v_credit;

END IF;

END LOOP label;

CLOSE youbiao;

END//

DELIMITER;

针对教学数据库创建一个存储函数,功能是根据课程号计算课程平均成绩,并将其作函数值返回

DELIMITER //

DROP FUNCTION IF EXISTS pro_avggrade //

CREATE FUNCTION pro_avggrade (v_cno CHAR(6))

RETURNS DECIMAL

READS SQL DATA

BEGIN

DECLARE avggrade DECIMAL(5,2);

SELECT AVG(grade) INTO avggrade FROM score WHERE cno = v_cno;

RETURN avggrade;

END//

DELIMITER;

针对教学数据库创建一个触发器,当修改成绩表中成绩时,要求将被修改成绩的学生学号、课程号、修改前的值和修改后的值保存到日志表中,该日志表由用户创建

```
CREATE TABLE update-log
(
  id UNSIGNED AUTO-INCREMENT PRIMARY KEY,
  sno CHAR(6),
  cno CHAR(3),
  valuebefore DECIMAL(3,1),
  valueafter DECIMAL(3,1),
  operate-date DATE,
  operate-user VARCHAR(20));

DELIMITER //
CREATE TRIGGER trig-score
AFTER UPDATE ON score
FOR EACH ROW
BEGIN
  INSERT INTO update-log (id, sno, cno, valuebefore, valueafter, operate-date, operate-user)
  VALUES (NULL, OLD.sno, OLD.cno, OLD.grade, NEW.grade, NOW(), USER());
END //
DELIMITER;
```

针对教学数据库创建一个数据表,字段包括id、dept、total、s-time. 创建一个周期性事件,该事件用于实现每周将每个系别的总人数插入创建的表

```
CREATE TABLE mytable
(
  id UNSIGNED AUTO-INCREMENT PRIMARY KEY,
  dept VARCHAR(20),
  total INT,
  s-time DATETIME);

DELIMITER //
DROP EVENT IF EXISTS myevent //
CREATE EVENT myevent
ON SCHEDULE EVERY 1 WEEK DO
BEGIN
  DECLARE v-dept VARCHAR(20);
  DECLARE v-number INT;
  DECLARE v-done BOOLEAN DEFAULT 0;
  DECLARE youbiao CURSOR FOR
  SELECT department, COUNT(*) FROM student GROUP BY department;
  DECLARE CONTINUE HANDLER FOR NOT FOUND SET v-done = 1;
  OPEN youbiao;
  label: LOOP
    FETCH youbiao INTO v-dept, v-number;
    IF v-done THEN LEAVE LOOP; END IF;
    INSERT INTO mytable (id, dept, total, stime)
    VALUES (NULL, v-dept, v-number, NOW());
  END LOOP label;
END //
DELIMITER;
```